# Modeling

PDF edition of preserved repository document | docs/modeling.md

Historical statements and test counts below retain their original scope. Consult the master report for current status.

# First learned prototype

## Hardware and model choice

Local inspection found an ARM64 CPU with 10 logical threads and NumPy 2.3.5.
The bundled Python runtime lacks PyTorch, Transformers, scikit-learn and TensorBoard.
A cached all-MiniLM-L6-v2 sentence encoder exists but is not an arithmetic generator
or action policy. RAM probing was denied by the sandbox; available RAM is unknown.
No GPU or download is required. The chosen model is a small task-specific NumPy MLP,
trained locally from seeded random initialization. It is not a foundation model or
language model. This is allowed by the discrete-classifier first-policy design.

Both models use the same 52-dimensional input space and 64-unit tanh hidden layer.
The answer-only baseline has eight ten-class digit heads (least-significant first,
zero padding) plus a two-class sign head: 8,722 trainable parameters. The trajectory
policy has four action logits: 3,652 parameters. The shared backbone has 3,392
parameters; output capacities differ and are recorded, not claimed identical.
Baseline feature slots for state/history remain zero. Neither encoder computes a
product. There are no stored lookup answers or pretrained proprietary dependencies.
Models support predict/save/load; the backend supports logits/generate/save/load.
Training is separate in training/, never in the benchmark or environment.

## State-conditioned policy

Canonical version policy-state-1.0 includes problem, state, goal, valid_actions and
previous_action. Numeric features encode operand digits/signs/lengths, positions,
carry, partial row/total digits, length boundaries and previous action. Feature
selection deliberately excludes the phase string and legal-action identities from
network inputs: they directly reveal the next action in the current environment.

The serialized record still preserves those fields for action validation/masking.

Goal has a fixed domain meaning; a learned goal encoder is not implemented.

Four learned logits score multiply/flush/accumulate/finish. By default illegal choices are masked, then the chosen action is sent to the authoritative environment. The actual event/result/state updates working memory. Prefix replay verifies every transition, and final replay/independent multiplication oracle verify termination. No generated explanation is used. Step and wall-clock limits terminate failures. The explanation interface is secondary and not implemented in this milestone.

## Critical identification limitation

The current environment exposes exactly one legal action in every nonterminal state. Masking therefore forces the correct procedural sequence, even with an untrained network. Masked 100% accuracy cannot establish learning, improved data efficiency, reasoning or generalization. The environment also supplies exact arithmetic unavailable to the answer-only baseline. This is an intentionally unmatched knowledge/tool condition, not a clean test of the research hypothesis.

We train the policy's unconstrained four-way cross entropy and report validation unmasked action accuracy. Additional unmasked episodes terminate on an illegal learned choice. These diagnostics exercise learned weights causally; an untrained masked control in tests confirms the confound. A meaningful future study needs multiple legal choices with consequences, matched tools/knowledge and proper controls. No superiority or DER conclusion is drawn from this prototype.

## Memory and regimes

Working memory resets per episode and stores state, goal, latest observed feedback, actions, events, actual results and trajectory prefix. SQLite episodic memory supports store/retrieve/get/clear with explicit partitions; retrieval defaults to train and cannot see test entries unless specifically requested. Model inference does not use cross-episode retrieval or automatically persist evaluation answers. The API is an experimental foundation, not a demonstrated memory learning signal.

Regime A trains only problem operands -> answer digits/sign. Regime B trains each trajectory state -> next action, with prior executed action as history. It is trajectory-supervised and becomes environment-interactive at inference. Regime C has an explicit feedback-learning interface raising NotImplementedError; no claim of experiential or self-learning behavior is made.

## Limits

Operand feature width is six decimal digits; answer width is eight. Larger inputs are rejected or unrepresentable. Training targets are range-checked; the included benchmark stays within those limits. Eight answer digits cannot cover every product of two six-digit operands. There is no neural arithmetic transition predictor, learned verifier, language understanding or transfer to a second domain. Plain NumPy checkpoints avoid pickle and reject nonfinite/wrong-shaped weights.

## Controlled numerical-claim revision

The revised study is documented in research_protocol.md and milestone_04.md. Prior singleton/oracle-assisted results are exploratory, not current evidence. Primary numerical-claim execution writes model proposals without correction; Regime C uses training-only scalar feedback and positive replay. All four conditions share capacity and the evaluator; remaining procedural/supervision/compute differences are explicit. All reported thresholds are censored; no data-efficiency advantage has been established. Use the research-specific CLI/config files for this study.